Listing 4.7 Managing the conversation history with `getMutableAIState`

```
export async function
continueConversation(input, provider, model) {
  'use server';
  const supportedModel = getSupportedModel(provider, model);
  const history = getMutableAIState();
  const result = await streamUI({.
    model:  supportedModel,
    messages: [...history.get(), { role: 'user', content: input }],
    text: ({ content, done }) => {
      if (done) {
        history.done([...history.get(),
{ role: 'assistant', content: input }]);
      }
      return <ChatBubble role="assistant" text={content}
className={`mr-auto border-none`} />;
    },
  });

  return {
    id: generateId(),
    role: 'assistant',
    display: result.value,
  };
}

export const AI = createAI({.
  actions: {
    continueConversation,
  },
  initialAIState: [],
  initialUIState: {
    messages: [],
  },
});
```

Gets the mutable AI state to access and update conversation history

Uses streamUI to generate a streaming response from the AI model

Combines the existing history with new user input for context

Updates the AI state when the response is complete

Returns the streamed UI component as the display value

Creates the AI context with the defined action and initial states

Notice that we didn't have to pass on the `history` object as a parameter. This `history` object represents the AI state that gets resolved on the server side using the `getMutableAIState` function. We can call this function anywhere on the server and get the conversation history.

That concludes all the backend updates. The following section deals with the necessary updates needed for the frontend UI.

CHANGES TO THE FRONTEND

To use our new state management system, we need to make some important changes to our frontend components. The AI component, which is created using the `createAI` function, is now set as the parent context provider for the entire application. This allows all child components to access the unified AI and UI state and actions. It wraps the application's layout with the `<AI>` component to provide its context to all descendant